Data Structures and Algorithms 2

CA Exercise 2 – National Gallery Route Finder

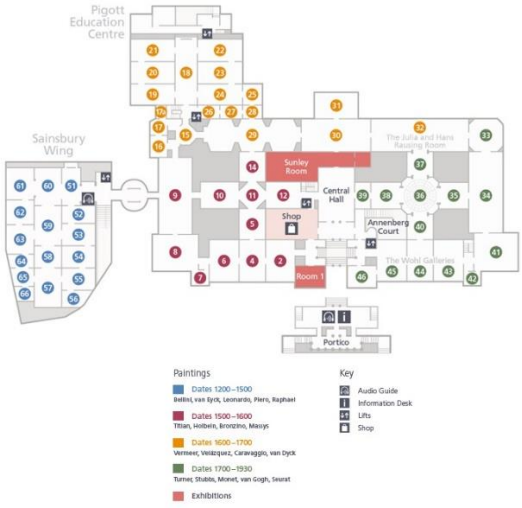
"Create a route finder for the main floor of the National Gallery, London."

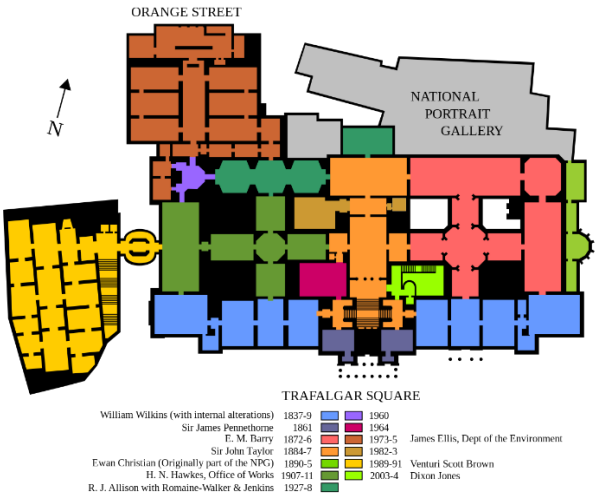
The objective of this team CA exercise is to create a JavaFX application that can search for and retrieve routes between points/rooms/exhibits on the main floor (level 2) in the National Gallery in London given a starting point/room/exhibit and a destination.

The application should create/generate the following on request:

- Multiple route permutations between a starting point/room/exhibit and a destination (can limit it to a maximum user-specified number of routes where there are too many permutations).
    - This functionality should use depth-first search (DFS).
- Shortest route (in terms of distance) between the starting point/room/exhibit and destination.
    - This functionality should come in two forms: one using Dijkstra's algorithm and one using breadth-first search (BFS).
- Most interesting route (based on a list of artists a visitor is particularly interested in) between the starting point/room/exhibit and destination.
    - Using Dijkstra's algorithm again.

The application should also allow users to specify rooms/exhibits to avoid on the route, and to also specify rooms/exhibits that must be visited on the route to the final destination. A suitable "blank" map/floorplan of the main floor of the National Gallery, London, should be used (use whatever map you like, and edit/fix/recolour it as appropriate). Some (unedited) examples:



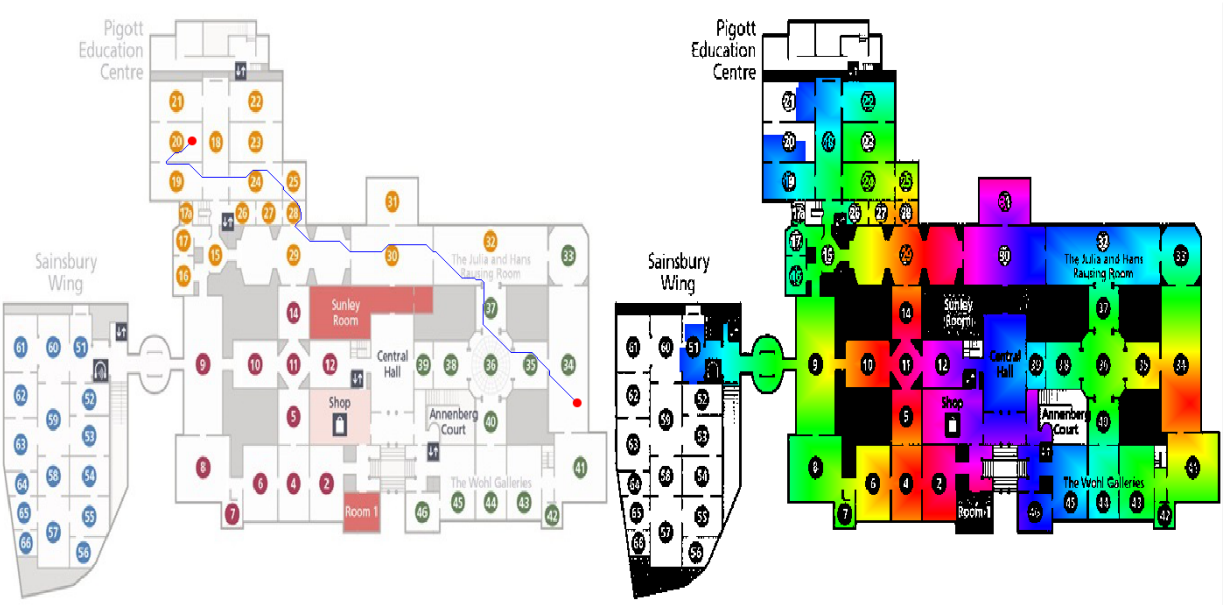


Lines, markers, and text labels, art thumbnails, etc. can be added to the map to graphically identify the route(s) as appropriate on the map.

Implementation Notes

- This is a team CA exercise. Teams should consist of 2 or 3 students only. Students may be permitted to work by themselves in very limited circumstances (this requires the prior approval of your lab lecturer).
- This CA exercise is worth 35% of your overall module mark. You must submit it on Moodle and demonstrate it (as a team) to your assigned lab lecturer for it to be assessed and included in your overall marks. The demo/interview is mandatory!
- The key point of this CA exercise is in the use of graphs (i.e. nodes/vertices connected by links/edges), along with some algorithms to search/traverse graphs.
    - Think of rooms as nodes/vertices and connecting doorways/corridors as links/edges. For instance, in the example map shown above (the one with numbers) room 60 connects to rooms 51, 59 and 61 only. Room 59 also connects to rooms 53, 58 and 63. So, to get from room 60 to 53, you could go 60-59-53 as one route (there are clearly other routes too).
        - Rooms will have art exhibits displayed in them, and these could be appropriately stored at the room nodes/vertices. Using an art exhibit as a starting point or destination will then just involve using the room nodes/vertices they are associated with.
    - You should develop your own custom graph-based data structure in the first instance as the main data structure for representing rooms/exhibits and their interconnecting doorways/corridors/etc. However, beyond the required custom graph, you are free to also use any other JCF collections/classes you wish to implement the required features and algorithms, etc.
- The system should be comprehensive and use a suitable database file of many rooms and exhibits in the National Gallery (enough to be useful for a real visitor, say).
    - The database can be stored or represented in any appropriate file format (e.g. CSV, XML, plaintext, etc.). There is no need to save back to the database file in the application, but it should be easy to update the database file manually to e.g. add new rooms (i.e. nodes/vertices) and connections (i.e. links/edges), and add new art exhibits to the rooms/nodes too.
    - You might find this interactive map useful for exhibit information.
- The application should provide a suitable user-friendly JavaFX graphical user interface.
    - Some level of "graphical feedback" should be provided on the map as part of the user interface (e.g. marking and linking points/rooms/exhibits along an identified route using lines). However, what the user interface looks like is ultimately up to you, and the key thing is the data graph representation and usage.
    - You could also use a TreeView control to show the route(s) as a collapsible hierarchy.
    - You could also use some suitable container/view (e.g. a scrolling list) to show images (along with titles and artists) of the art exhibits encountered along a route.
    - Other JavaFX controls might also be used as appropriate to provide for a nice, intuitive, and coherent user experience.
- You should use Dijkstra's algorithm to identify the shortest (and/or most interesting, based on a specified list of artists for a visitor) route between two known rooms/exhibits (i.e. those with entries in the database).

- You should use breadth-first search (BFS) to identify a route between any two "pixel points" in any accessible room or area in the National Gallery. In this case, the BFS proceeds "pixel-by-pixel" to establish the shortest route between the two points. Using a black-and-white version of your map whereby rooms/doorways/corridors/atriums are (say) white (note you can just do this B&W conversion in a paint package as the map image is fixed): adjacent pixels to the starting point (pixel) can be considered to see if they are also white and, if so, they are treated as children and added to the agenda list (as usual with BFS). Distances to children (i.e. adjacent pixels) can be deemed unary (i.e. always have a value of 1 unit). The BFS continues until the destination point (pixel) is reached. The BFS route should be illustrated by a line on the map and the estimated distance reported. Example from rooms 34 to 20 (estimated distance of 590 units/pixels given the map size/scale used):



  Note that your result/output should resemble the image on the above left-hand side. The right-hand side image simply shows the BFS pattern using a hue-shift (from 0-to-360, or red-to-magenta, cyclic) to illustrate the pixel/node expansion order breadth first. You are not expected to reproduce the right-hand image (though it is straightforward when BFS is working).
- A route may have specified waypoints (one or more rooms/exhibits) between the starting point and destination that the route(s) must go through. Waypoints should ideally be supported in some way in all route-finding operations (single route, multiple routes, shortest route, most interesting route, etc.)
    - Note: waypoints do not have to be supported in the BFS approach.
- Similarly, it should also be possible to specify rooms/exhibits to avoid on the route.
    - Note: not applicable to the BFS approach.

Indicative Marking Scheme

- Custom graph data structure/classes = 10%
- Generate any single valid route between two points/rooms/exhibits = 10%
- Generate multiple valid route permutations using DFS = 10%
- Shortest route using Dijkstra's algorithm = 15%
- Shortest route using breadth-first search (BFS) (and illustrating route on map) = 15%
- Most interesting route using Dijkstra's algorithm = 10%
- Waypoint support = 5%
- Avoiding specified rooms/exhibits = 5%
- JavaFX GUI = 5%
- JUnit testing = 5%
- JMH benchmarking of key methods = 5%
- General (overall completeness, structure, commenting, logic, etc.) = 5%

Note that it is not expected that all students will attempt all aspects of this assignment. Use the above marking scheme to guide your efforts as this is what you will be marked against.